

30-Day FreeRTOS Course for ESP32 Using ESP-IDF (Day 16)

“Task Notifications”



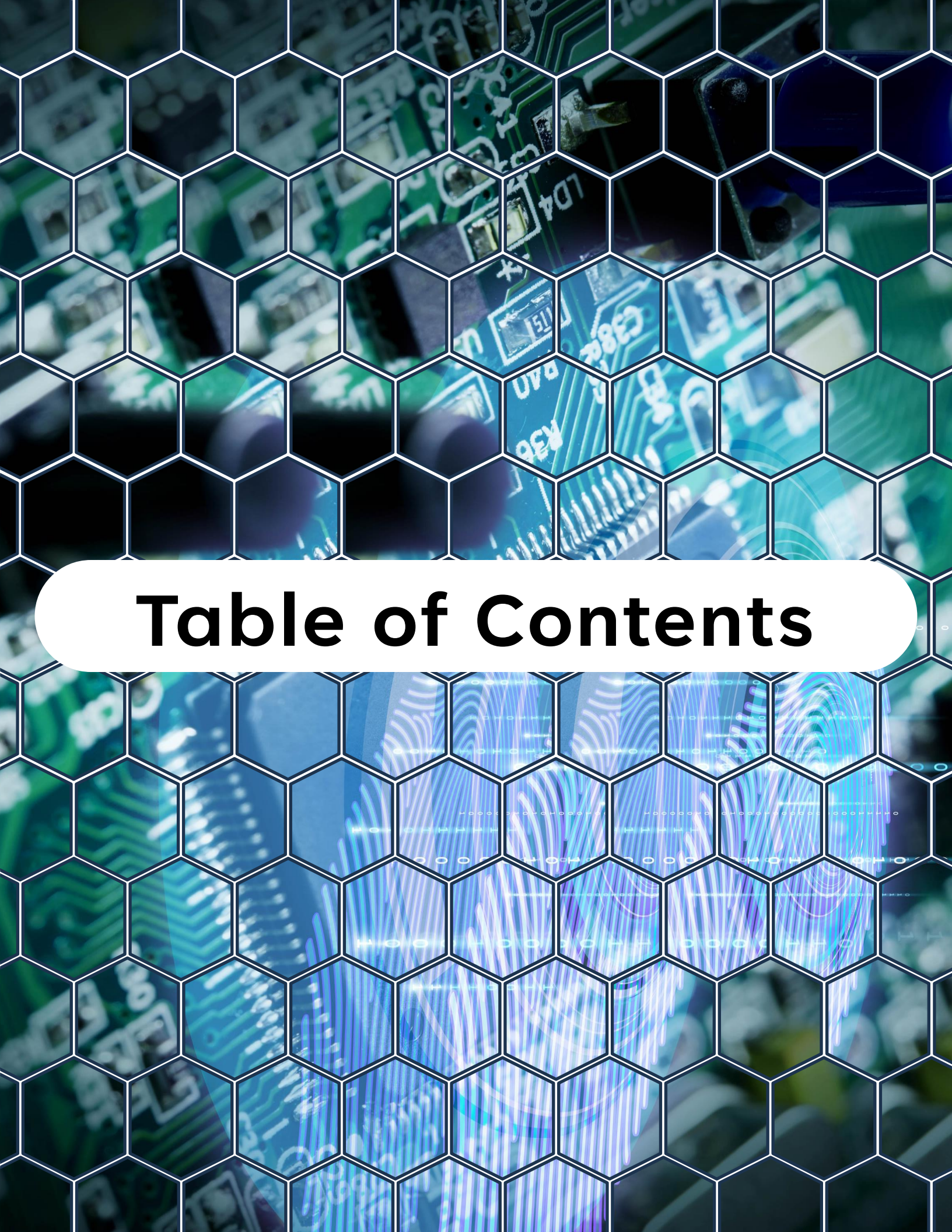
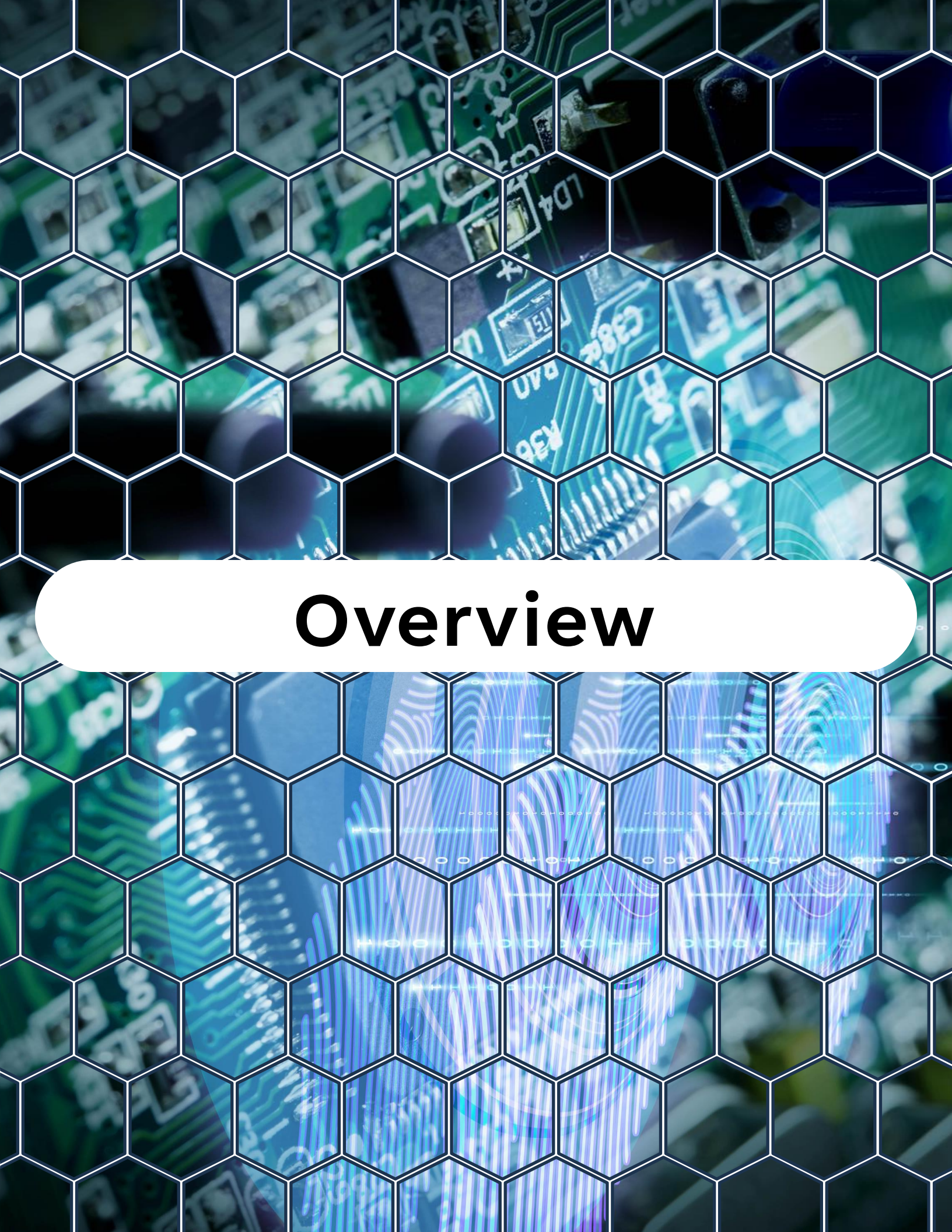


Table of Contents

Table of Contents

1. Overview
2. What Are Task Notifications?
3. Task Notification Functions
4. Notify Actions
5. Why Use Task Notifications?
6. Example – ISR Triggering a Task with Notifications
7. Best Practices
8. Summary
9. Challenge for Today



Overview

Overview

On **Day 16**, you'll learn:

- What **task notifications** are in FreeRTOS
- How they compare to semaphores and event groups
- Different modes of task notification (overwrite, increment, bitwise, etc.)
- How to send notifications from tasks and ISRs
- A practical ESP32 example using **ISR** → **Task Notification**
- Best practices for task notification usage

By the end of this lesson, you'll know how to use task notifications as a lightweight, efficient way to synchronize tasks and ISRs.



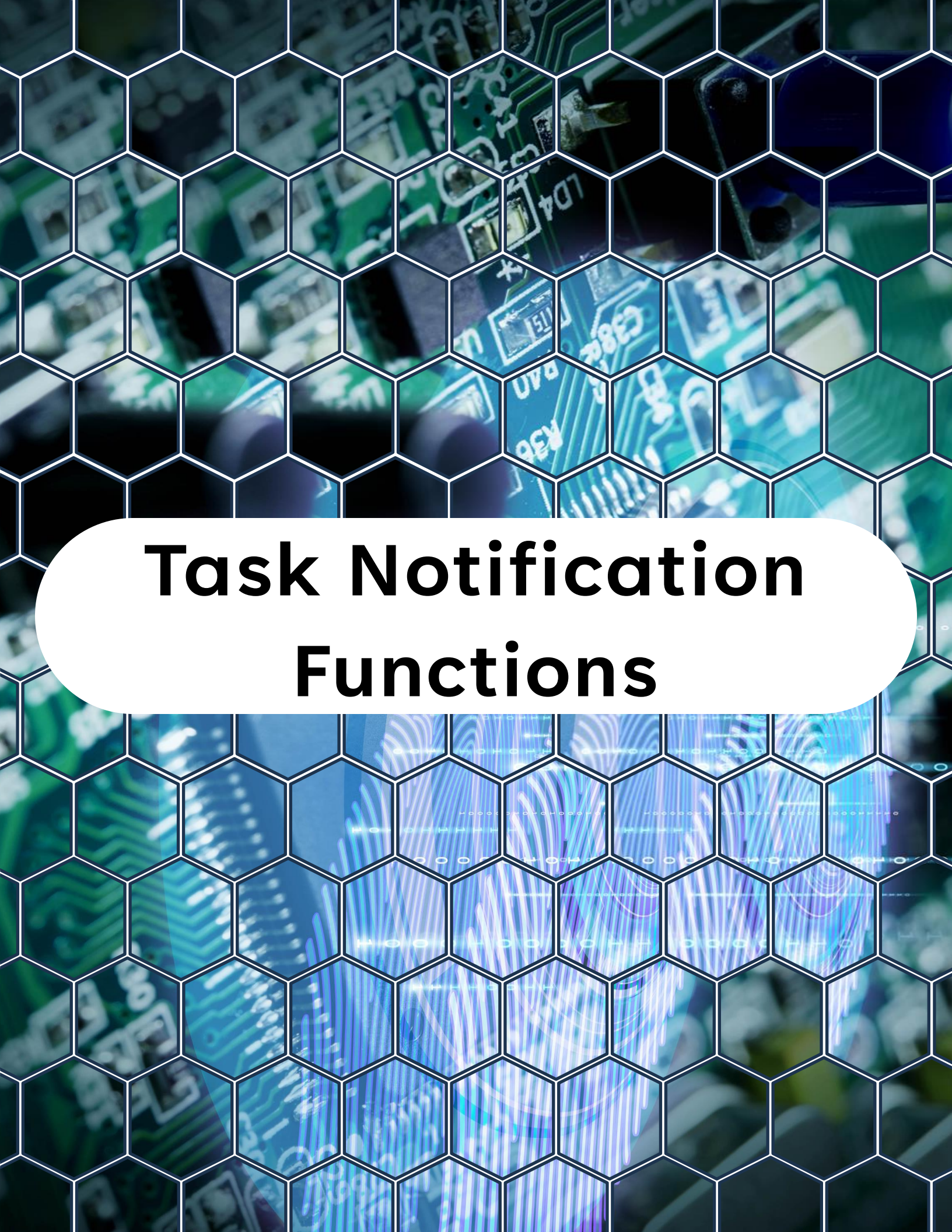
What Are Task Notifications?

What Are Task Notifications?

A **task notification** is a built-in synchronization mechanism in FreeRTOS.

- Each task has a **notification value** (like a private mailbox).
- Another task or ISR can **send a notification** to that task.
- The notified task can **read, clear, or wait** on its notification.

 Think of it as a **lightweight, one-to-one signaling system**, faster and more memory-efficient than semaphores or queues.



Task Notification Functions

Task Notification Functions

Sending Notifications

From another task:



```
1 xTaskNotify(TaskHandle_t xTaskToNotify,  
2             uint32_t ulValue,  
3             eNotifyAction eAction);
```

From ISR:



```
1 xTaskNotifyFromISR(TaskHandle_t xTaskToNotify,  
2                    uint32_t ulValue,  
3                    eNotifyAction eAction,  
4                    BaseType_t *pxHigherPriorityTaskWoken);
```

Receiving Notifications



```
1 xTaskNotifyWait(uint32_t ulBitsToClearOnEntry,  
2                uint32_t ulBitsToClearOnExit,  
3                uint32_t *pulNotificationValue,  
4                TickType_t xTicksToWait);
```



Notify Actions

Notify Actions

eNotifyAction defines how the notification value is updated:

- **eNoAction** → just unblock the task (don't change value).
- **eSetBits** → set bits (bitwise OR).
- **eIncrement** → increment value.
- **eSetValueWithOverwrite** → overwrite with new value.
- **eSetValueWithoutOverwrite** → update only if no pending notification.

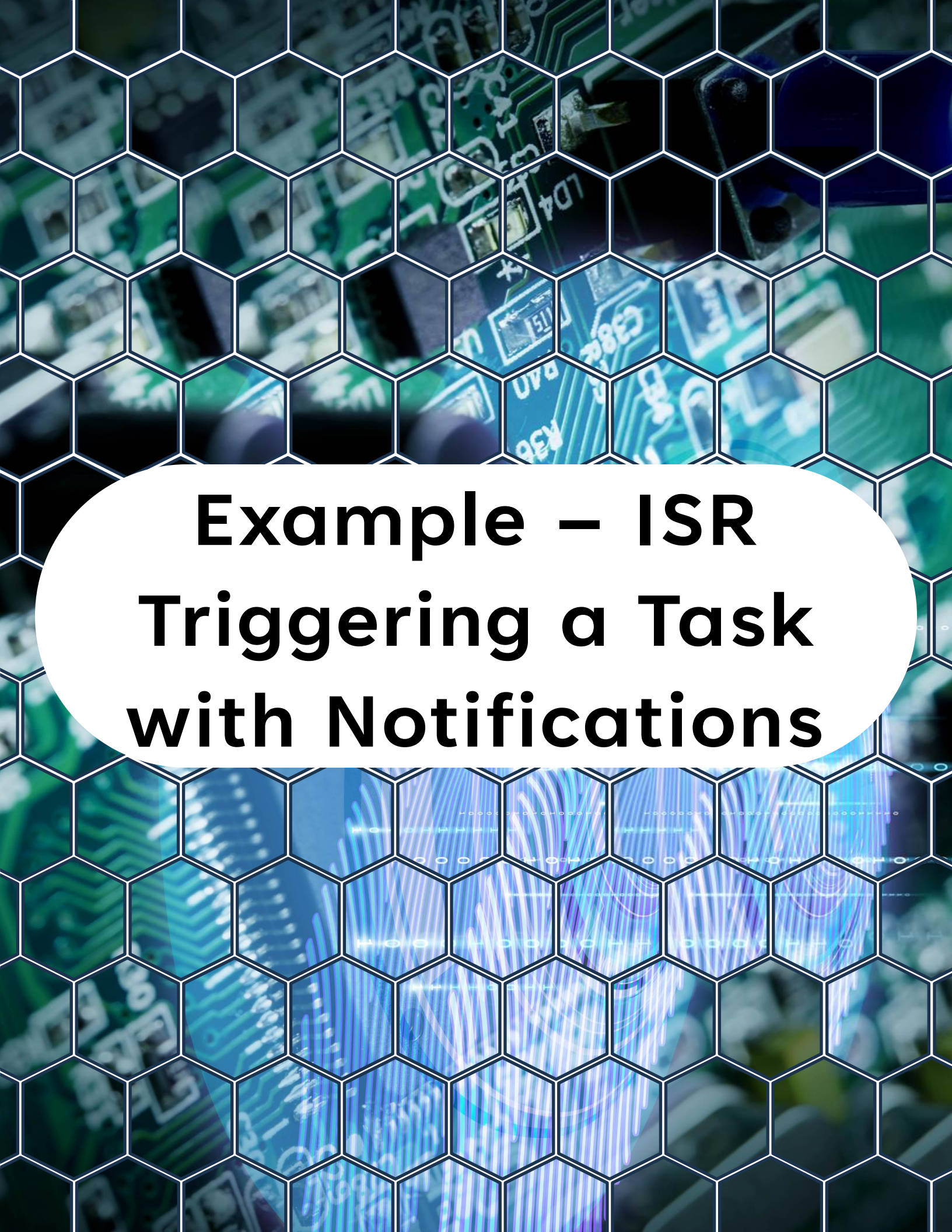


Why Use Task Notifications?

Why Use Task Notifications?

- **Lightweight** → No need to allocate memory (unlike queues).
- **Fast** → Directly signals the task, fewer instructions than semaphores.
- **Flexible** → Supports counting, bit flags, or simple signaling.
- **One-to-one** → Ideal for ISR → Task or Task → Task communication.

 If you only need to signal **one task**, task notifications are the most efficient option.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the main title text.

Example – ISR Triggering a Task with Notifications

Example – ISR Triggering a Task with Notifications

```
1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "driver/gpio.h"
5  #include "esp_log.h"
6
7  #define BUTTON_GPIO 0
8  #define TAG "DAY16"
9
10 // Task handle for the button task to allow
11 // notification from ISR
12 TaskHandle_t button_task_handle = NULL;
13
14 // ISR handler
15 static void IRAM_ATTR button_isr_handler(void *arg) {
16     // Variable to check if a higher priority task
17     // was woken by the notification
18     BaseType_t xHigherPriorityTaskWoken = pdFALSE;
19
20     // Notify the button task
21     vTaskNotifyGiveFromISR(button_task_handle,
22                             &xHigherPriorityTaskWoken);
23
24     // Yield to higher priority task if necessary
25     if (xHigherPriorityTaskWoken) {
26         portYIELD_FROM_ISR();
27     }
28 }
29
30 // Task waiting on notifications
31 void button_task(void *pvParameter) {
32     while (1) {
```

Example – ISR Triggering a Task with Notifications

```
29
30 // Task waiting on notifications
31 void button_task(void *pvParameter) {
32     while (1) {
33         // Wait indefinitely for a notification
34         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
35         ESP_LOGI(TAG, "Button pressed! Task unblocked by ISR");
36     }
37 }
38
39 // Main application entry point
40 void app_main() {
41     // Configure button pin
42     gpio_config_t io_conf = {
43         .intr_type = GPIO_INTR_NEGEDGE,
44         .mode = GPIO_MODE_INPUT,
45         .pin_bit_mask = (1ULL << BUTTON_GPIO),
46         .pull_up_en = GPIO_PULLUP_ENABLE,
47         .pull_down_en = GPIO_PULLDOWN_DISABLE
48     };
49     gpio_config(&io_conf);
50
51     // Install ISR service and attach handler
52     gpio_install_isr_service(0);
53     gpio_isr_handler_add(BUTTON_GPIO, button_isr_handler, NULL);
54
55     // Create the button task
56     xTaskCreate(button_task, "ButtonTask", 2048, NULL, 10,
57                 &button_task_handle);
58 }
```


Example – ISR Triggering a Task with Notifications

Expected Behavior

- Press the button on GPIO0.
- The ISR runs and **gives a notification** to the waiting task.
- The `button_task` unblocks and logs a message.

Output:

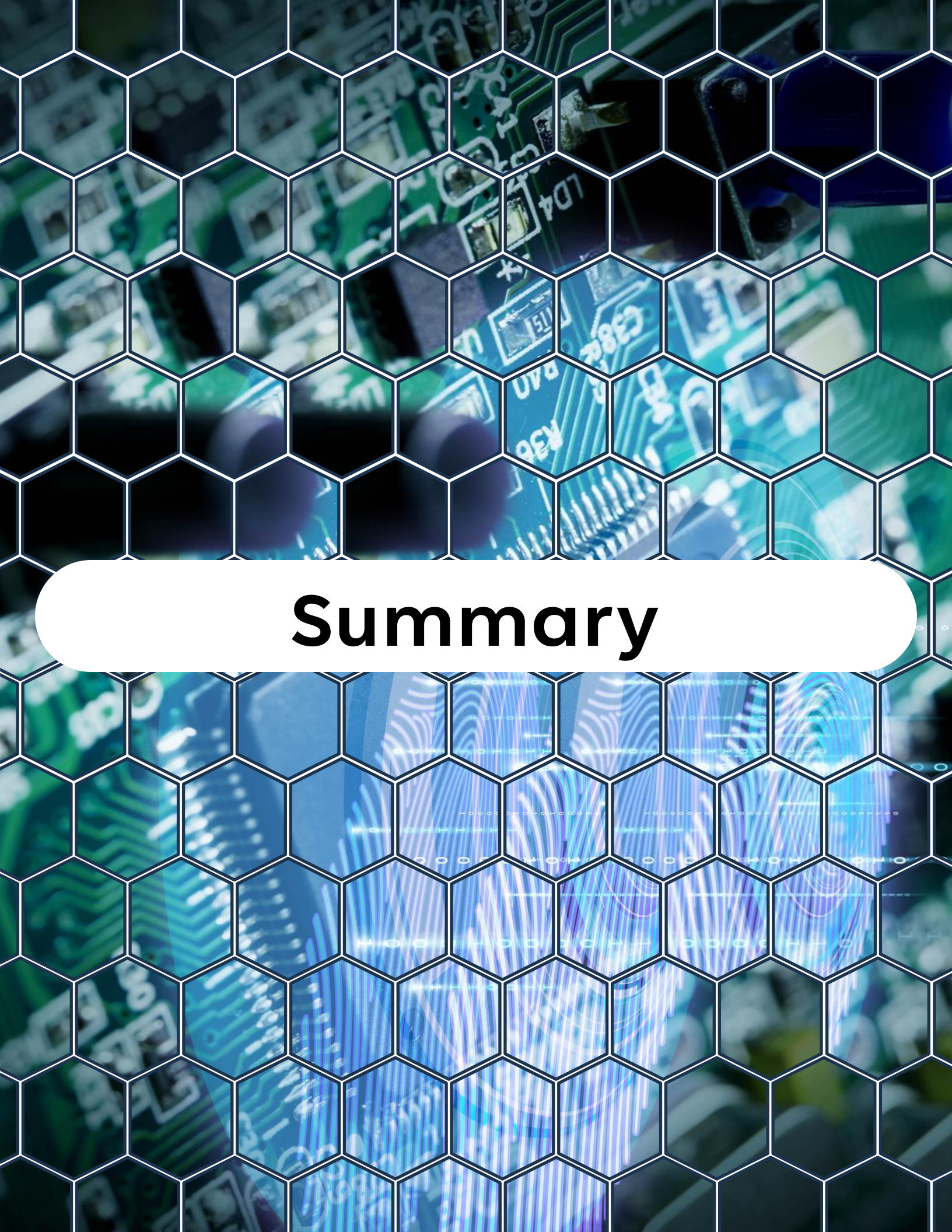
I (1234) DAY16: Button pressed! Task unblocked by ISR



Best Practices

Best Practices

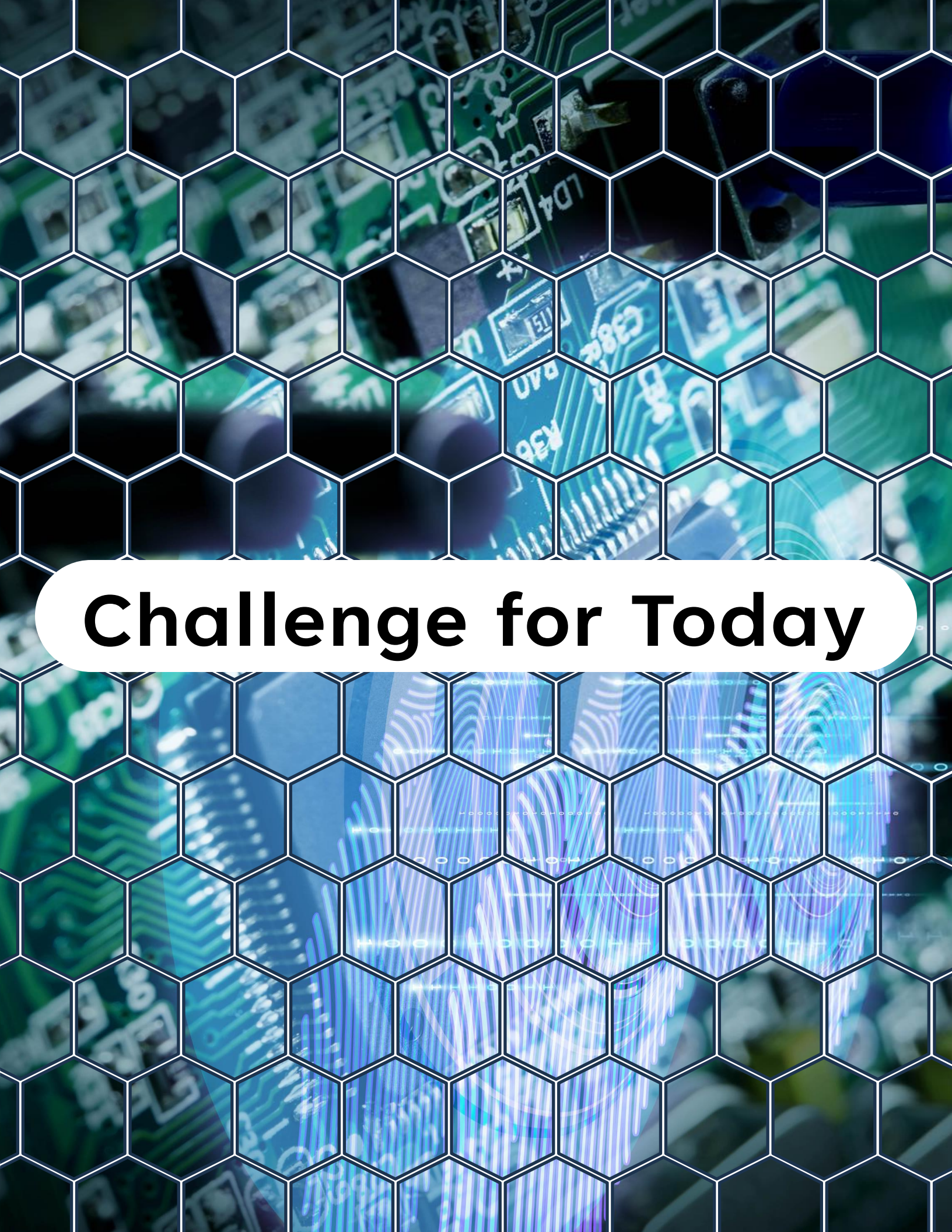
- ✓ Use **task notifications** for **one-to-one signaling** (ISR → Task, Task → Task).
- ✓ Use **queues** if you need to pass **data** or multiple messages.
- ✓ Use **event groups** if **multiple tasks** need to be synchronized.
- ✓ Use `ulTaskNotifyTake()` for counting events (like button presses).
- ✓ Keep ISRs short: only notify, don't do logic inside.



Summary

Summary

- Task notifications are the **fastest, lightest** signaling mechanism in FreeRTOS.
- Each task has a built-in notification value.
- Notifications can increment, set bits, overwrite, or just signal.
- Perfect for **ISR** → **Task** signaling where performance matters.



Challenge for Today

Challenge for Today



Modify today's project by:

- Counting the number of button presses using `ulTaskNotifyTake(pdTRUE, ...)`.
- Print a running counter in the task (e.g., “Button pressed 5 times”).
- Bonus: Use `xTaskNotify()` from another task to simulate button presses.

"Thanks for watching!

If you enjoyed this video,

*make sure to hit the like button and
subscribe to stay updated with my latest
content.*

*Don't forget to check out my other
videos for more tips and tutorials on
Embedded C, Python, hardware designs,
etc. Keep exploring, keep learning, and
I'll see you in the next video!"*



<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>